COLÉGIO UNASP

DSW – AV1 e AV2 – 4º Bimestre

Valor: AV1 – 5,0; AV2 – 5,0

Data final de entrega das duas partes: 27/11/26.

Forma de entrega: apresentação em sala de aula para o professor.

Acesso rápido às avaliações:

- AV1 – Desenvolvimento do backend – 5,0
- AV2 – Frontend e publicação – 5,0

1 Objetivo

Desenvolver uma aplicação web full stack para atender a uma necessidade concreta. O mesmo projeto será desenvolvido em duas etapas: backend completo na AV1 e frontend integrado com publicação na AV2.

2 Tema e tecnologias

O aluno poderá escolher o tema, mantendo um escopo viável para o bimestre. As sugestões abaixo são pontos de partida; não é necessário implementar todas elas.

Sugestões de temas

Tema	**Cadastro principal e funcionalidades**	**Exemplo de regra de negócio**
Organizador de tarefas e estudos	Cadastrar tarefas com disciplina, prazo, prioridade e situação	Impedir a conclusão de uma tarefa enquanto houver pendências vinculadas a ela
Agenda de serviços	Cadastrar agendamentos com serviço, responsável, data e horário	Impedir agendamentos com horários sobrepostos para o mesmo responsável
Controle de estoque	Cadastrar produtos e registrar entradas e saídas	Impedir saídas maiores que a quantidade disponível
Biblioteca e empréstimos	Cadastrar livros e registrar empréstimos e devoluções	Impedir o empréstimo de um exemplar que já está emprestado

Tema	Cadastro principal e funcionalidades	Exemplo de regra de negócio
Reserva de equipamentos	Cadastrar equipamentos e reservar períodos de uso	Impedir reservas sobrepostas para o mesmo equipamento
Inscrição em oficinas	Cadastrar oficinas e gerenciar inscrições	Impedir novas inscrições quando a capacidade da oficina for atingida
Controle de despesas pessoais	Cadastrar despesas por categoria, mês e situação de pagamento	Impedir alterações em despesas de um mês já encerrado pelo usuário
Acompanhamento de chamados técnicos	Cadastrar chamados com descrição, prioridade e situação	Permitir encerrar um chamado somente após registrar a solução aplicada

Em qualquer tema, o aluno deverá definir quem pode consultar, criar, alterar e excluir os registros. Por exemplo: cada usuário gerencia apenas suas próprias tarefas ou um administrador gerencia as oficinas, enquanto os demais usuários realizam inscrições.

O escopo poderá ser limitado a um processo principal. Não são exigidos pagamentos on-line, envio de e-mails, upload de imagens ou integrações externas. Funcionalidades adicionais são opcionais e não substituem os requisitos da avaliação.

Tecnologias e requisitos comuns

A aplicação deverá conter um cadastro principal com operações completas de criação, consulta, atualização e exclusão (CRUD), cadastro e autenticação de usuários, controle de acesso e pelo menos uma regra de negócio coerente com o tema.

Tecnologias obrigatórias:

- Backend: Node.js e Express, com arquitetura MVC.
- Acesso ao banco: Prisma ORM, obrigatoriamente na versão **7.10**, com os pacotes `prisma` e `@prisma/client` fixados em **7.10.0**, sem `^` ou `~`.
- Banco de dados relacional.
- Autenticação: JWT e armazenamento de senhas com hash.
- Frontend: React e Tailwind CSS, obrigatoriamente na versão **4.3**, com o pacote `tailwindcss` fixado em **4.3.0**, sem `^` ou `~`. Integrações oficiais do Tailwind utilizadas no projeto deverão acompanhar a mesma versão.

É incentivado utilizar componentes Tailwind prontos disponíveis na internet, como formulários, tabelas, menus, cartões e modais. O aluno deverá verificar a compatibilidade com React e Tailwind CSS 4.3, respeitar as licenças, adaptar os componentes ao projeto e registrar as fontes no README. Os componentes deverão funcionar com os dados reais da aplicação.

Validações de campos obrigatórios e formatos são necessárias, mas não substituem a regra de negócio. Exemplos de regras: impedir agendamentos em horários ocupados; impedir uma saída superior ao estoque disponível; limitar o número de tarefas em andamento.

3 AV1 – Desenvolvimento do backend – 5,0

Voltar ao acesso rápido

Requisitos

1. Criar uma API em Node.js com Express, organizada em arquitetura MVC, separando rotas, controllers, regras de negócio e acesso aos dados. Nesta etapa, a API fornecerá respostas JSON; a interface em React será desenvolvida na AV2.
2. Modelar o banco relacional e integrá-lo pelo Prisma ORM 7.10, mantendo schema e migrations no repositório.
3. Implementar CRUD completo do cadastro principal, com persistência real no banco de dados.
4. Validar os dados no backend, utilizando códigos HTTP e mensagens adequadas.
5. Implementar pelo menos uma regra de negócio, com verificação no backend.
6. Implementar cadastro de usuários e login, armazenando senhas com hash e emitindo JWT após autenticação válida.
7. Criar middleware para validar o JWT e proteger as rotas necessárias.
8. Implementar autorização coerente com o projeto: por exemplo, permitir que cada usuário altere apenas seus próprios registros ou restringir determinadas operações por perfil.
9. Tratar erros e utilizar variáveis de ambiente para credenciais do banco e segredo JWT.
10. Entregar README com instruções para instalar dependências, configurar o ambiente, preparar o banco e executar a API, além da descrição das principais rotas. Incluir `.env.example` sem valores secretos.

Entrega e apresentação em laboratório

Entregar o link do repositório com o backend funcional. A API poderá ser executada localmente e deverá ser demonstrada com Postman, Insomnia ou ferramenta equivalente. Não será exigido frontend nesta etapa.

O aluno deverá:

- Explicar o objetivo do projeto, a estrutura do backend e a modelagem do banco.
- Demonstrar o CRUD e a persistência dos dados.
- Testar uma situação aceita e outra bloqueada pela regra de negócio.
- Demonstrar cadastro, login e uso do JWT.
- Demonstrar o bloqueio de uma requisição sem autenticação e de uma operação não autorizada.
- Explicar trechos do código solicitados pelo professor.

Critérios de avaliação

Os critérios foram reorganizados para concentrar a avaliação do backend na AV1.

Critério	**Evidências esperadas**	**Peso**	**Nota máxima**
Backend, banco e estrutura MVC	Organização da API, modelagem e integração com Prisma 7.10	25%	1,25
CRUD e persistência	Operações completas, validação dos dados e gravação no banco	20%	1,00
Autenticação, autorização e proteção de rotas	Cadastro, hash de senhas, login, JWT, middleware e restrições de acesso	25%	1,25
Implementação de regra de negócio	Regra pertinente, aplicada e demonstrada no backend	20%	1,00
Organização, documentação e domínio do backend	Tratamento de erros, configuração, README e explicação do código	10%	0,50
Total		**100%**	**5,00**

4 AV2 – Desenvolvimento do frontend e publicação – 5,0

Voltar ao acesso rápido

Requisitos

1. Desenvolver o frontend em React, utilizando Tailwind CSS 4.3.
2. Criar uma interface organizada, responsiva e coerente com o tema. É incentivado adaptar componentes Tailwind prontos da internet.
3. Integrar o frontend ao backend desenvolvido na AV1, utilizando dados reais da API.
4. Disponibilizar pela interface as operações completas do CRUD.
5. Criar telas de cadastro e login, integrar a autenticação por JWT e implementar logout.
6. Controlar o acesso às páginas que exigem autenticação e apresentar as ações conforme as permissões do usuário.
7. Tratar sessão inválida ou expirada, erros de validação, acesso negado e falhas de comunicação.
8. Apresentar mensagens de sucesso e erro, estados de carregamento e indicação de listas vazias.
9. Publicar frontend e backend, utilizando banco de dados acessível ao backend em produção. O fluxo completo deverá funcionar pela URL pública. Como sugestão, utilizar o **Render**

para hospedar o backend e a **Vercel para hospedar o frontend**. Outras plataformas poderão ser utilizadas, desde que atendam aos requisitos de publicação do projeto.

10. Configurar as variáveis de ambiente de cada ambiente, sem expor credenciais ou segredos no frontend ou no repositório.
11. Atualizar o README com as tecnologias e versões, instruções de execução de frontend e backend, preparação do banco, variáveis necessárias sem valores secretos, fontes dos componentes utilizados e endereço da aplicação publicada.

A proteção implementada no React deverá complementar a autenticação e a autorização da API. As verificações de segurança do backend, desenvolvidas na AV1, deverão permanecer funcionando.

Entrega e apresentação em laboratório

Entregar o link do repositório atualizado e o endereço da aplicação publicada. A apresentação será individual, ao professor, em laboratório.

O aluno deverá contextualizar o projeto, explicar o problema identificado, o público-alvo e a solução proposta. Em seguida, deverá demonstrar pela aplicação publicada:

- Cadastro, login e logout.
- CRUD completo integrado ao backend e ao banco.
- Aplicação da regra de negócio.
- Controle de acesso conforme autenticação e permissões.
- Tratamento de situações de erro.

Durante a apresentação, deverá explicar componentes React, chamadas à API, uso do Tailwind e decisões de implementação.

Critérios de avaliação

Os critérios foram reorganizados para concentrar a avaliação do frontend, da integração e da publicação na AV2.

Critério	**Evidências esperadas**	**Peso**	**Nota máxima**
Estrutura React e integração frontend/backend	Componentes organizados, comunicação com a API e uso de dados reais	25%	1,25
CRUD e regra de negócio pela interface	Operações completas e retorno adequado das validações e regras	20%	1,00
Integração da autenticação e controle de acesso	Cadastro, login, logout, páginas protegidas e tratamento da sessão	20%	1,00

Critério	Evidências esperadas	Peso	Nota máxima
Deploy funcional de frontend, backend e banco de dados	Fluxo completo disponível e funcionando pela URL pública	25%	1,25
Interface, documentação e domínio do projeto	Tailwind 4.3, usabilidade, responsividade, README e explicação do código	10%	0,50
Total		**100%**	**5,00**

5 Orientações gerais

- O projeto e as apresentações serão individuais.
- A AV2 será a continuação do mesmo projeto apresentado na AV1.
- O repositório deverá estar atualizado e acessível ao professor em cada entrega, incluindo os arquivos de dependências e o lockfile.
- O aluno deverá compreender o código entregue, inclusive componentes prontos e trechos adaptados de materiais externos ou produzidos com ferramentas de apoio.
- Credenciais, senhas e segredos não deverão ser incluídos no repositório.
- A avaliação considerará o funcionamento demonstrado e as evidências correspondentes a cada critério.